10_16_aprile

Continuo lezione 9_14_aprile

Concetto di Priorità

Introduciamo il concetto di **priorità**: si assegna la CPU al processo di priorità più alta. Viene così creata una **coda di priorità**.

- I processi a priorità bassa possono essere eseguiti se nella coda non ci sono processi a priorità più alta.
- La **prelazione** può essere applicata a questo meccanismo: se un processo ad alta priorità entra in coda ma la CPU è assegnata a un processo a bassa priorità, qui avviene la prelazione.

L'assegnazione della priorità avviene tramite diverse motivazioni (es. fattori esterni)

- **Priorità statiche**: Dovute a fattori esterni come utente admin o argomenti vari.
- **Priorità dinamiche**: Il sistema può decidere di cambiare la priorità a determinati processi. Come per esempio lo scheduler decide di dare priorità ai processi **I/O bounded** invece che ai **CPU bounded**. Appena la risposta I/O arriva, i processi I/O bounded che erano bloccati tornano in coda pronti e sono con priorità alta.

Per capire se un processo è I/O o CPU bounded bisogna osservare come usano il loro **quanto di tempo Q**. Il processo I/O di solito si blocca prima perché aspetta un device di I/O, mentre quello CPU se lo prende tutto.

Potrei quindi pensare di mappare la priorità sull'uso di un unico quanto di tempo: se un processo usa tutto il Q allora avrà priorità minima, meno Q usa più priorità avrà.

$$P(prio) = Q - Q_{usato}$$

(Quantizzato a valori interi minimi)

La priorità qui è dinamica poiché se un processo cambia l'uso di Q avrà priorità diversa.

- Un altro modo sarebbe sfruttare il **SJF** facendo una stima della durata dei processi e assegnando più priorità a durata minore e meno priorità a durata maggiore.
- *Nei sistemi interattivi è meglio applicare la prelazione.*

La **starvation** si presenta quando non posso mai garantire a un processo in coda (bassa priorità) l'uso immediato della CPU.

Per risolvere questo problema posso usare l'**aging**: esso aumenta temporaneamente la priorità dei processi che trascorrono troppo tempo nella coda dei processi pronti; più

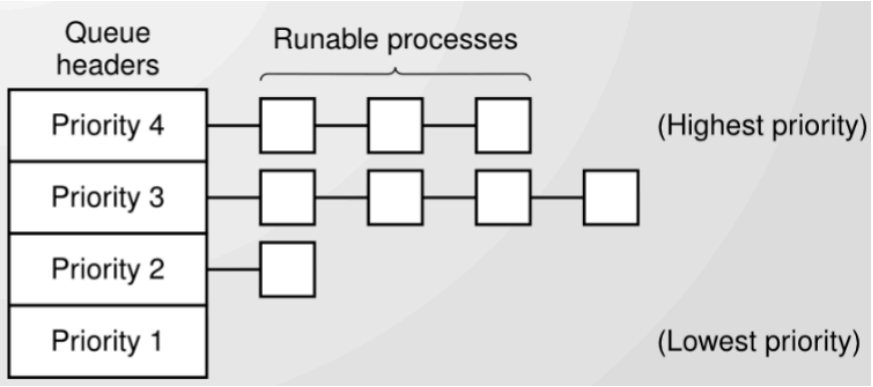
tempo aspetta, più aumenta di priorità. Dopo che viene eseguito, la priorità torna a quella di base.

Scheduling a Code Multiple

Potremmo usare un sistema di **scheduling a code multiple**: avrò una coda per ogni livello di priorità, in ogni coda avrò processi della stessa priorità.

Bisogna avere due scheduling diversi che scelgono quale processo far eseguire:

1. **Scheduling verticale**: Viene scelta la prima coda non vuota dall'alto (priorità dall'alto al basso).
2. **Scheduling orizzontale**: Scelta la coda, bisogna scegliere un processo tra quelli (insieme di uguale priorità). Utilizzo un algoritmo visto in precedenza come il **Round Robin (FIFO)**. Potrei avere diversi algoritmi di scheduling per ogni coda, potrei anche applicare $RR(Q)$ con Q variabile da un livello all'altro.



Metto a priorità alta processi I/O bounded e bassa quelli CPU bounded in mezzo processi misti.

Avrò quanti di tempo più piccoli nelle priorità alte dato che ci sono processi I/O bounded e rendono il sistema più reattivo, e nelle priorità basse un quanto più grande.

> **Nota visiva (piramide delle code):**
>
> - $Q_1 \leftarrow Prio4$ (Vertice)
> - $Q_2 \leftarrow Prio3$
> - $Q_3 \leftarrow Prio2$
> - $Q_4 \leftarrow Prio1$ (Base)
> - La durata Q aumenta scendendo verso il basso.
> - Posso assegnare a Q_4 un quanto infinitamente grande o un FCFS.

Per evitare la starvation posso realizzare un **up/down grade** della priorità in base all'uso del suo quanto di tempo. Potrei fare una media su come il processo usa il suo Q: se lo usa tutto, Q è troppo piccolo, quindi è CPU bounded e allora scenderà di un livello.

Un altro modo è cambiare lo scheduling verticale assegnando un tempo T a un ciclo. Per ogni livello verrà assegnata una percentuale di T a ogni coda di priorità. Se una coda è vuota la percentuale di T sarà minore (es. *50%, 25%, 15%, 10%*).

SPN Shortest Process Next e Stima del CPU Burst

Applico lo **SJF** ai processi interattivi. In questi sistemi devo però capire la durata del prossimo burst di CPU. Posso fare delle stime dei burst precedenti di quel processo dato che il burst di un processo è simile ai burst di quelli precedenti.

CPU Burst: Durata del tempo da quando il processo riceve la CPU a quando si blocca per I/O o timeout.

Questo sistema è senza quanti di tempo.

$S =$ Stima (do più valore ai valori recenti)

$$S_{n+1} = aS_n + (1-a)T_n$$

- S_n: Vecchia stima
- T_n: Ultima misurazione

Così la nuova stima considera l'ultima misurazione e la vecchia stima dando un determinato peso a T_n e S_n. Il peso viene dato proprio da a che rappresenta una costante $0 < a < 1$ che in base al suo valore scelto si sceglie se far valere di più la vecchia stima o l'ultima misurazione.
(es $a = 0.5$ in questo caso avrò che S_n e T_n avranno lo stesso peso)

Altri tipi di Scheduling

- **Scheduling Garantito**: Viene stabilita una percentuale di utilizzo per ogni processo e viene fatta rispettare. Bisogna quindi fare delle promesse ai processi e vedere poi se sono state rispettate. Chi è più indietro con le promesse avrà una percentuale più alta; va a dare la CPU a chi l'ha usata di meno. Non è basato sul quanto di tempo.
- **Scheduling a Lotteria (Gambling)**: Assegno un tot numero di biglietti ai processi, estraggo un biglietto ed eseguo il processo a cui ho assegnato il biglietto. Dopo che lo estraggo brucio il biglietto. Se un processo finisce i biglietti non vedrà più la CPU.

Quando i biglietti finiscono si parte da 0. I biglietti chiaramente possono essere assegnati tramite vari criteri (es processi I/O bounded assegno più biglietti rispetto che CPU bounded)

- **Scheduling Fair-Share**: In un sistema multi-utente avrò utenti con pochi processi e utenti con molti. Un sistema Fair Share dovrebbe garantire equità tra gli utenti e non solo tra i processi nei sistemi operativi.
  - *Esempio*: Utenti U_1, U_2. $U_1 = 4$ processi, $U_2 = 1$ processo.
  - Se il sistema è equo sui processi avrò: U_1 usa l'80% delle risorse, U_2 il 20%.